

Sistema de Escalas para Igrejas

Versão simplificada: PHP puro + JavaScript + MySQL + Docker

Guia do que é preciso para rodar + briefing completo do projeto

Referência de preços e regras: 01/10/2026. Documento revisado em 04/10/2026.

Parte 1: contas, ferramentas, ambiente com Docker, servidor, custos e checklist inicial.

Parte 2: briefing para entregar a outra IA continuar o planejamento (decisões, regras de negócio, arquitetura, notificações, IA e projeto de referência).

O que mudou em relação à versão anterior: saem o Laravel, o Blade, o Livewire, o Alpine.js, o Composer, o Node e o Redis. Entram PHP puro (sem framework), JavaScript simples, MySQL, Bootstrap e Docker (apenas no desenvolvimento). O motivo é o nível de experiência da equipe: menos peças, mais fáceis de entender e de achar ajuda. As funcionalidades e as regras de negócio continuam as mesmas.

Parte 1: O que você precisa para rodar o sistema

Resumo prático do que precisa estar pronto (contas, ferramentas e servidor) para desenvolver e colocar o sistema no ar. Itens marcados com **(conferir)** dependem de preços ou requisitos que mudam com o tempo e devem ser verificados na fonte oficial antes de contratar.

1. Visão geral em uma frase

Um servidor com PHP e MySQL (hospedagem compartilhada ou VPS), com domínio e HTTPS, e-mail transacional e WhatsApp Business Platform (API oficial) para os avisos. O desenvolvimento roda em Docker no seu computador.

Escopo atual: cerca de 200 pessoas, e só o que é necessário agora. Sem framework, sem Redis, sem Node. Os avisos automáticos usam uma tabela de fila no MySQL e um script PHP chamado por cron a cada minuto.

2. Contas e serviços a contratar

Item	Para que serve	Observação
Domínio	Endereço do sistema (ex.: escalas.suaigreja.com.br)	Registro.br para .com.br. Cobrança anual (conferir)
Hospedagem	Roda o sistema	Opção A (mais simples): hospedagem compartilhada com PHP 8.x, MySQL, cron e SSL grátis. Confirmar versão do PHP, frequência mínima do cron e envio de e-mail (conferir). Opção B: VPS Linux com Docker (mais controle, mais manutenção). Comparar provedores (conferir)
HTTPS	Segurança do login	Certificado gratuito (Let's Encrypt), normalmente já incluso na hospedagem compartilhada
E-mail transacional	Avisos e redefinição de senha	Amazon SES, Brevo ou similar, via SMTP. Verificar o domínio (SPF, DKIM e DMARC no DNS) para não cair em spam
Meta Business verificada	Exigida pela API oficial do WhatsApp	Verificação da organização, que pode pedir documentos e demorar (conferir)
WhatsApp Business Platform (Cloud API) ou provedor	Envio dos avisos	Decidir: direto na Meta (sem mensalidade de plataforma, mais trabalho técnico) ou via provedor como Zenvia ou Twilio. Conferir taxas em reais
Número de telefone	Remetente das mensagens	O número da igreja já está no aplicativo do WhatsApp. Confirmar com a Meta/provedor como registrá-lo na API
Forma de pagamento	Cobrança das mensagens	Conforme Meta ou provedor (conferir)
Repositório Git privado	Versionar o código e ter cópia	GitHub ou GitLab
Destino de backup	Cópia do banco fora do servidor	Armazenamento externo ou rotina de backup da hospedagem (conferir custo)

3. Ferramentas no computador de desenvolvimento

- **Docker Desktop** (no Windows, com WSL2). É a única instalação pesada.
- **Git**
- **Editor de código** (VS Code, por exemplo)
- **Navegador** (Chrome ou Firefox)

- **Não precisa instalar** PHP, MySQL, Node ou Composer no computador: tudo roda dentro dos contêineres.
- Opcional: biblioteca PHPMailer (instalada via Composer, dentro do contêiner) para enviar e-mails por SMTP de forma confiável. A função mail() do PHP costuma cair em spam (conferir).

4. Estrutura do projeto

```
escalas/
|-- docker-compose.yml
|-- Dockerfile
|-- db/
|   +-- schema.sql      (cria as tabelas na primeira subida)
+-- src/                (todo o código PHP e JavaScript)
    |-- config.php      (dados do banco e chaves; NAO vai para o Git)
    |-- conexao.php     (conecta no banco com PDO)
    |-- regras.php      (TODAS as regras de escala ficam aqui)
    |-- login.php
    |-- minhas_escalas.php (voluntário vê onde está escalado)
    |-- bloqueios.php    (voluntário bloqueia dias)
    |-- trocas.php
    |-- lider_escala.php  (líder monta a escala)
    |-- cron_avisos.php  (roda a cada minuto e envia mensagens)
    |-- includes/
    |   |-- topo.php     (menu, repetido em todas as páginas)
    |   +-- rodape.php
    +-- assets/
        +-- app.js       (JavaScript simples: confirmações, janelas)
```

5. Ambiente com Docker

Dockerfile

```
FROM php:8.3-apache

RUN docker-php-ext-install pdo pdo_mysql

# Fuso horário do Brasil (importante para datas e lembretes)
ENV TZ=America/Sao_Paulo
RUN ln -snf /usr/share/zoneinfo/$TZ /etc/localtime && echo $TZ > /etc/timezone
```

docker-compose.yml

```
services:
  web:
    build: .
    ports:
      - "8080:80"
    volumes:
      - ./src:/var/www/html
    depends_on:
      - db

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root_dev
      MYSQL_DATABASE: escalas
      MYSQL_USER: escalas
      MYSQL_PASSWORD: escalas_dev
      TZ: America/Sao_Paulo
    volumes:
      - dados_mysql:/var/lib/mysql
      - ./db/schema.sql:/docker-entrypoint-initdb.d/schema.sql
    command: --character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci

  phpmyadmin:
    image: phpmyadmin:latest
    ports:
      - "8081:80"
    environment:
      PMA_HOST: db
      PMA_USER: root
      PMA_PASSWORD: root_dev
    depends_on:
      - db

volumes:
  dados_mysql:
```

Primeiros comandos

```
docker compose up -d                # sobe PHP, MySQL e phpMyAdmin
# Sistema:      http://localhost:8080
# phpMyAdmin:  http://localhost:8081

# testar o envio de avisos uma vez:
docker compose exec web php /var/www/html/cron_avisos.php

# simular o cron (a cada 60 segundos):
docker compose exec web sh -c "while true; do php /var/www/html/cron_avisos.php; sleep 60; done"

docker compose down                # desliga, mantendo os dados
```

Conexão com o banco (src/conexao.php)

Dentro do Docker, o endereço do banco é o **nome do serviço** (db), não localhost:

```
<?php
$pdo = new PDO(
    'mysql:host=db;dbname=escalas;charset=utf8mb4',
    'escalas',
    'escalas_dev',
    [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]
);
```

Cuidados com o Docker: as senhas acima são **só para desenvolvimento**. Nunca exponha o phpMyAdmin na internet. O comando *docker compose down -v* apaga o banco de teste (use só *down* no dia a dia). O *schema.sql* só roda na primeira criação do banco; mudanças posteriores exigem recriar o volume ou aplicar pelo phpMyAdmin.

6. Exemplo de tela completa: bloqueios.php

Cada tela é um arquivo PHP com o HTML dentro. O PHP no topo faz o trabalho e o HTML embaixo mostra o resultado:

```
<?php
session_start();
require 'conexao.php';

if (!isset($_SESSION['usuario_id'])) {
    header('Location: login.php');
    exit;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // (verificar o token CSRF aqui, ver seção de segurança)
    $sql = "INSERT INTO bloqueios (usuario_id, data_inicio, data_fim) VALUES (?, ?, ?)";
    $pdo->prepare($sql)->execute([$_SESSION['usuario_id'], $_POST['data_inicio'], $_POST['data_fim']]);
}

$stmt = $pdo->prepare("SELECT * FROM bloqueios WHERE usuario_id = ?");
$stmt->execute([$_SESSION['usuario_id']]);
$bloqueios = $stmt->fetchAll();
?>
<?php include 'includes/topo.php'; ?>

<h1>Meus bloqueios</h1>
<form method="POST">
    <input type="date" name="data_inicio" required>
    <input type="date" name="data_fim" required>
    <button type="submit">Bloquear</button>
</form>

<ul>
<?php foreach ($bloqueios as $b): ?>
    <li><?= htmlspecialchars($b['data_inicio']) ?> até <?= htmlspecialchars($b['data_fim']) ?></li>
<?php endforeach; ?>
</ul>

<?php include 'includes/rodape.php'; ?>
```

JavaScript: usado só em pequenos detalhes, em *src/assets/app.js*, por exemplo pedir confirmação antes de excluir:

```
// no HTML: <button data-confirma="Excluir este bloqueio?">Excluir</button>
document.querySelectorAll('[data-confirma]').forEach(function (el) {
    el.addEventListener('click', function (e) {
        if (!confirm(el.dataset.confirma)) e.preventDefault();
    });
});
```

7. O que configurar em produção

Opção A: hospedagem compartilhada

- Enviar a pasta *src/* por FTP/SFTP (ou Git, se a hospedagem permitir) e importar o *schema.sql* pelo phpMyAdmin da hospedagem.
- Criar o *config.php* de produção direto no servidor, com senhas fortes (fora do repositório).

- Cadastrar no painel da hospedagem um cron a cada minuto: `php /caminho/cron_aviso.php` (conferir se a hospedagem permite intervalo de 1 minuto).
- Ativar HTTPS e forçar o redirecionamento de http para https.
- Desativar a exibição de erros na tela (`display_errors` desligado) e registrar em arquivo de log.

Opção B: VPS com Docker

- Usar o mesmo `docker-compose.yml`, com senhas novas e **sem** o phpMyAdmin exposto.
- Colocar HTTPS na frente (Caddy ou Nginx com Let's Encrypt, com renovação automática).
- Cron do servidor chamando o script de avisos a cada minuto.
- Firewall liberando só SSH, HTTP e HTTPS; acesso por chave SSH; atualizações de segurança em dia.

Nas duas opções

- Backup diário do banco para fora do servidor e teste periódico de restauração.
- Monitoramento simples (aviso quando o site cair).
- Chaves de API e senhas fora do Git (`config.php` no `.gitignore`).

8. Os cinco cuidados de segurança (obrigatórios)

Sem framework, a segurança fica por conta do código. São poucas regras, mas não podem ser puladas:

1. **SQL injection:** sempre usar `prepare()` e `?` nas consultas. Nunca juntar variáveis direto no texto do SQL.
2. **XSS:** sempre usar `htmlspecialchars()` ao mostrar dados na tela.
3. **Senhas:** `password_hash()` para salvar e `password_verify()` para conferir. Nunca guardar senha em texto puro.
4. **Permissão em toda página:** checar o perfil (administrador, líder, voluntário) no começo de cada arquivo, antes de qualquer ação.
5. **CSRF:** todo formulário que altera dados deve levar um token secreto guardado na sessão e conferido no servidor. Também chamar `session_regenerate_id(true)` após o login.

9. Custos mensais (orçamento: cerca de R\$ 150)

Item	Estimativa	Observação
Hospedagem (conferir)	a definir	Compartilhada costuma custar menos que VPS. Comparar provedores antes de contratar
WhatsApp	cerca de R\$ 20 a 35	Estimativa para cerca de 500 a 800 mensagens de utilidade por mês (200 pessoas, 2 a 3 escalas cada, mais trocas), com base na tabela de referência de US\$ 0,0068 por mensagem no Brasil. Conferir a taxa em reais com o provedor
E-mail (conferir)	a definir	Muitos serviços têm faixa gratuita
Domínio	cobrança anual (conferir)	Registro.br para <code>.com.br</code>
Certificado HTTPS	Grátis	Let's Encrypt

Com 200 pessoas o WhatsApp cabe com folga no orçamento. Se o sistema crescer para mais de 1.000 voluntários, a estimativa sobe para algo como R\$ 80 a 120 por mês; nesse caso, usar WhatsApp apenas para o lembrete (ex.: 48h antes) e para as trocas, e deixar o restante por e-mail e painel.

10. Checklist antes de começar o código

- [] Escolher e contratar a hospedagem (compartilhada ou VPS)
- [] Registrar o domínio
- [] Criar a conta Meta Business e iniciar a verificação (pode demorar)
- [] Confirmar como registrar na API o número que já está no WhatsApp
- [] Escolher o serviço de e-mail e verificar o domínio
- [] Criar o repositório Git privado
- [] Instalar o Docker Desktop e testar o docker compose up
- [] Definir o administrador geral e alguns líderes para teste
- [] Reunir as escalas atuais (planilhas, PDFs) para a importação inicial
- [] Redigir o texto de consentimento e a política de privacidade (LGPD)
- [] Entregar a Parte 2 deste documento à IA que ajudará no código

11. Anexo: como rodar o projeto de referência "Escala 12" (apenas para testar)

1. Instale o Docker Desktop.
2. Na raiz do projeto, execute docker compose up.
3. Abra <http://localhost:5173>.
4. Login inicial: usuário admin, senha admin123 (trocar antes de qualquer uso real).
5. A API fica em <http://localhost:3001> e a documentação (Swagger) em <http://localhost:3001/docs>.
6. Para e-mails reais, configure SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS e SMTP_FROM no serviço do backend. Sem isso, as notificações ficam registradas só no banco.

Estes passos vêm do README do próprio projeto; o sistema não foi executado. O README do backend avisa que o código não pôde ser instalado no ambiente em que foi gerado, então podem aparecer erros de pacote na primeira execução.

Parte 2: Briefing do projeto (Sistema Web de Gestão de Escalas de Igreja)

Como usar esta parte: cole o texto desta Parte 2 na primeira mensagem de uma nova conversa com outra IA (ou anexe este PDF). Ele resume tudo o que foi decidido até aqui (referência: 04/10/2026). A IA deve assumir o papel de arquiteto/mentor técnico do projeto e continuar de onde paramos.

1. Instruções para a IA que receber este arquivo

- Responda sempre em português do Brasil.
- Não suponha nada. Se faltar informação, pergunte antes de decidir (no máximo 3 a 5 perguntas por vez, objetivas).
- O time tem 2 pessoas com **pouca experiência**. Explique passo a passo, com exemplos de código prontos para copiar, e indique onde cada arquivo deve ficar.
- **Não usar frameworks nem bibliotecas pesadas:** nada de Laravel, Blade, Livewire, Alpine.js, React, Vue ou Node. O usuário pediu algo mais simples. Usar PHP puro e JavaScript simples.
- Prefira soluções simples, baratas e fáceis de manter.
- Informações que mudam com o tempo (principalmente preços da API do WhatsApp) devem ser conferidas na fonte oficial antes de se tornarem decisão. Os números abaixo são de referência.

2. Contexto do projeto

- Sistema somente web (sem aplicativo nativo) para gestão de escalas de voluntários de igrejas.
- Tecnologias definidas: **PHP e JavaScript**, sem framework. Docker no desenvolvimento.
- Hoje: cerca de 200 pessoas usarão o sistema. O planejamento inicial considerava mais de 1.000 voluntários, distribuídos em várias igrejas da mesma organização ("tudo junto", com administração central). Decisão atual do usuário: construir apenas o que for necessário agora.
- Cada igreja tem vários departamentos, e uma mesma pessoa pode atuar em departamentos diferentes.
- Cada voluntário é escalado em média 2 a 3 vezes por mês.
- Orçamento: cerca de R\$ 150 por mês (hospedagem + mensagens).
- Equipe: 2 pessoas, pouca experiência com PHP. O usuário sabe usar Docker.
- Hospedagem ainda não definida (compartilhada ou VPS).
- A igreja já tem um número de telefone que já está em uso no aplicativo do WhatsApp.

3. Funcionalidades definidas

1. O líder monta a escala manualmente (decisão atual).
2. Login por e-mail e senha para todos os voluntários.
3. O voluntário pode: visualizar os dias em que está escalado; bloquear dias em que não pode servir (férias, doença, viagem etc.); solicitar troca diretamente com outro membro.
4. Notificações por WhatsApp (principal) e e-mail (complemento).
5. Geração de escala assistida por algoritmo e, mais tarde, IA (ver seção 9).

Perfis de acesso (proposta): administrador geral, administrador da igreja, líder de departamento e voluntário. No banco, uma coluna perfil na tabela de usuários; cada página confere o perfil no início.

4. Regras de negócio

1. **Ninguém pode atuar em 2 dias seguidos no mesmo departamento.** "Dias seguidos" significa dias do calendário (exemplo: sábado e domingo).
2. **Ninguém pode estar em 2 cargos/funções no mesmo dia e horário.**
3. **Bloqueios de data** informados pelo voluntário devem ser respeitados ao escalar.
4. **Troca de escala:** se a troca violar alguma regra de negócio, o sistema não a rejeita automaticamente: envia para aprovação do líder.

Fluxo sugerido da troca (proposta, ainda não confirmada pelo usuário)

- O voluntário pede a troca; o destinatário aceita ou recusa.
- No momento do aceite o sistema revalida tudo com as regras (a situação pode ter mudado).
- Se estiver tudo certo, a troca é aplicada dentro de uma transação (beginTransaction/commit do PDO).
- Se violar alguma regra, a troca passa ao estado AGUARDANDO_LIDER e só é aplicada se o líder aprovar.
- Estados: PENDENTE, ACEITA, RECUSADA, AGUARDANDO_LIDER, APROVADA_PELo_LIDER, REJEITADA_PELo_LIDER.
- Ponto a detalhar com o usuário: quem exatamente aprova (líder do departamento?), o que acontece se o líder recusar, e se há prazo de validade do pedido.

Arquitetura das regras: todas ficam em um único arquivo (src/regras.php), usado pela montagem manual, pelas trocas e pelo gerador automático. Exemplo conceitual:

```
<?php
// src/regras.php
function violacoes(PDO $pdo, int $usuarioId, array $evento, int $depId): array
{
    $erros = [];

    if (temBloqueio($pdo, $usuarioId, $evento['data'])) {
        $erros[] = 'Voluntário bloqueou esta data.';
    }
    if (escaladoDiaAnteriorOuSeguinte($pdo, $usuarioId, $depId, $evento['data'])) {
        $erros[] = 'Já atua neste departamento em dia seguido.';
    }
    if (temOutraFuncaoNoMesmoHorario($pdo, $usuarioId, $evento)) {
        $erros[] = 'Já está escalado em outra função neste horário.';
    }

    return $erros; // vazio = pode escalar
}
```

5. Arquitetura técnica recomendada

Camada	Decisão	Motivo
Backend	PHP 8.3 puro, sem framework, com PDO	Simples de entender; hospedagem barata. A segurança depende de seguir os cinco cuidados (Parte 1, seção 8)
Frontend	HTML + Bootstrap + JavaScript simples	Bootstrap entrega visual responsivo sem escrever CSS do zero. JavaScript só para detalhes (confirmações, janelas). As páginas recarregam a cada ação
Banco de dados	MySQL 8 (ou MariaDB), utf8mb4	Comum em hospedagem barata, muito material para aprender. Usar chaves estrangeiras e transações
Fila de envios	Tabela fila_envios no MySQL + cron_aviso.php	Substitui fila e agendador de framework. Sem Redis
Agendador	Cron a cada minuto chamando cron_aviso.php	Envia pendentes e cria lembretes (ex.: 48h antes)
Hospedagem	Compartilhada (PHP + MySQL) ou VPS com Docker	Cabe no orçamento. Provedor ainda não escolhido; comparar preços antes
Desenvolvimento	Docker (PHP/Apache + MySQL + phpMyAdmin)	Ambiente igual para os dois membros da equipe
Backup	Diário, do banco de dados	Segurança dos dados
Acesso mobile	Site responsivo (Bootstrap), opcionalmente PWA	Maioria dos voluntários usará o celular

Organização do código

- Uma tela por arquivo PHP (login.php, minhas_escalas.php, bloqueios.php, trocas.php, lider_escala.php...).
- includes/topo.php e includes/rodape.php repetidos em todas as páginas (menu e layout).
- regras.php com todas as regras de escala; funcoes de notificação em um único arquivo (notificar.php), para trocar de canal ou provedor sem reescrever o resto.
- Disciplina de pastas desde o início: o código de PHP puro pode ficar bagunçado se crescer sem organização.

Modelo multi-igreja

Um único sistema e um único banco, com coluna igreja_id nas tabelas principais. Cada igreja só enxerga seus dados; a administração geral enxerga tudo. Toda consulta deve filtrar por igreja_id conforme o perfil do usuário logado.

Modelo de dados (proposta inicial)

- **igrejas**
- **departamentos** (pertencem a uma igreja)
- **usuarios** (com coluna perfil e senha_hash)
- **departamento_usuario** (permite a mesma pessoa em vários departamentos, com a função dela em cada um)
- **eventos**
- **escalas** (usuário + evento + departamento + status)
- **bloqueios** (usuário + data início + data fim)
- **trocas** (escala de origem, solicitante, destinatário, status, validade, indicador de aprovação do líder quando houver violação de regra)

- **auditoria** (quem fez, ação, entidade afetada, dados extras em JSON, data/hora): registra geração de escala, edições manuais, trocas e aprovações. Preenchida por uma função chamada nas telas
- **notificacoes** (usuário, título, mensagem, canal, lida/não lida)
- **fila_envios** (destinatário, canal, mensagem/template, status PENDENTE/ENVIADO/ERRO, tentativas, agendado_para)
- Status da escala por período/departamento: RASCUNHO, ENVIADA (publicada aos voluntários), ACEITA

LGPD

- Dados que revelam vínculo religioso são dados sensíveis.
- Registrar consentimento para receber mensagens e limitar quem acessa as listas de contatos.
- Nos bloqueios, guardar apenas "indisponível" no período, sem o motivo (doença é dado de saúde; férias/viagem não precisam ser registradas).

6. Notificações

Decisão: WhatsApp (API oficial da Meta) como canal principal, e-mail como complemento/reserva. SMS descartado (mais caro e menos útil).

- Não usar APIs não oficiais (conexão por QR Code): violam os termos do WhatsApp e há risco de banimento do número.
- É possível integrar direto com a API oficial da Meta (sem mensalidade de plataforma, mais trabalho técnico) ou via provedor (ex.: Zenvia, Twilio), que simplifica, mas pode cobrar à parte. A chamada é feita por HTTP (cURL do PHP).
- Mensagens proativas usam templates pré-aprovados (ex.: "Você foi escalado para X no dia Y").
- Fluxo de envio: o sistema grava a mensagem na tabela fila_envios; o cron_avisos.php, a cada minuto, lê os pendentes, envia, marca como ENVIADO ou ERRO e tenta de novo os que falharam (limite de tentativas). O mesmo script cria os lembretes automáticos (ex.: 48h antes).
- Tudo passa por uma camada única de notificações, para trocar de canal/provedor sem reescrever o sistema.
- Evitar que duas execuções do cron enviem a mesma mensagem: marcar o registro como EM_ENVIO antes de enviar.

```
<?php
// src/cron_avisos.php (esqueleto)
require 'conexao.php';
require 'notificar.php';

$pendentes = $pdo->query(
    "SELECT * FROM fila_envios
      WHERE status = 'PENDENTE' AND agendado_para <= NOW() AND tentativas < 3
      ORDER BY id LIMIT 50"
)->fetchAll();

foreach ($pendentes as $item) {
    $pdo->prepare("UPDATE fila_envios SET status='EM_ENVIO' WHERE id=?")->execute([$item['id']]);
    $ok = enviarMensagem($item); // WhatsApp ou e-mail, conforme $item['canal']
    $pdo->prepare("UPDATE fila_envios SET status=?, tentativas=tentativas+1 WHERE id=?")
        ->execute([$ok ? 'ENVIADO' : 'PENDENTE', $item['id']]);
}
```

Custos (referência, conferir na fonte oficial)

- Desde 01/07/2025 a Meta cobra por mensagem-modelo entregue.
- Referência para o Brasil: mensagem de utilidade cerca de US\$ 0,0068; marketing cerca de US\$ 0,0625. Avisos de escala se enquadram em utilidade. Isso equivale a algo na casa de 4 centavos de

real por aviso, variando com câmbio e margem do provedor.

- Cobrança em reais disponível para contas no Brasil desde 01/07/2026.
- A partir de 01/10/2026, mensagens de utilidade enviadas dentro de uma janela de 24h já aberta deixam de ser gratuitas.
- Estimativa de volume: com 2 a 3 escalas por pessoa por mês e usando o WhatsApp só para lembretes e trocas, com cerca de 200 pessoas são 500 a 800 mensagens/mês (R\$ 20 a 35 pelo preço de referência; estimativa a confirmar com o provedor). Se chegar a mais de 1.000 voluntários, o custo sobe para algo como R\$ 80 a 120 por mês.

Estratégia para controlar custo

- WhatsApp apenas para: lembrete da escala (ex.: 48h antes), pedido de troca recebido, resposta da troca.
- E-mail e painel do sistema para: publicação de escala nova, avisos gerais, bloqueios.

Ponto em aberto importante

O número da igreja já está em uso no aplicativo do WhatsApp. É preciso verificar com a Meta/provedor como registrá-lo na API oficial: o registro tradicional exige um número que não esteja no aplicativo (pode ser necessário migrar ou usar um recurso de convivência entre app e API, se disponível). Não confirmado ainda. Também é necessária a verificação da empresa na Meta (Meta Business).

7. Etapas de construção sugeridas

1. Ambiente Docker + banco (schema.sql) + cadastros (igrejas, departamentos, pessoas, perfis) + login por e-mail e senha.
2. Eventos e escala manual pelo líder, com o regras.php validando e avisando sobre bloqueios e conflitos. Já registrar a auditoria das alterações.
3. Área do voluntário: ver escalas e bloquear dias.
4. Trocas entre voluntários (com a mesma validação; violação leva à aprovação do líder).
5. Notificações por e-mail e depois WhatsApp, com lembretes automáticos (cron + fila_envios).
6. Gerador automático de rascunho de escala.
7. (Opcional, depois) Pedido em texto com IA de linguagem.

Escopo agora (MVP), com cerca de 200 pessoas: etapas 1 a 5. A etapa 6 entra em seguida, quando o básico estiver estável. A etapa 7 e as ideias de IA da seção 10 ficam para depois.

8. Segurança (resumo para a IA)

Toda sugestão de código deve seguir: prepare() com parâmetros; htmlspecialchars() na saída; password_hash()/password_verify(); verificação de perfil e de igreja_id no início de cada página; token CSRF em todo formulário que altera dados; session_regenerate_id(true) no login; erros não exibidos na tela em produção; config.php fora do Git.

9. Sobre "IA para gerar as escalas"

O usuário quer incluir IA para gerar as escalas. Posicionamento acordado:

- Montar escala é um problema de regras e restrições. Um modelo de linguagem (LLM) não deve ser o mecanismo principal: pode errar regras, variar o resultado a cada execução e gera custo por uso.
- **Camada 1, gerador automático (algoritmo, em PHP):** percorre as vagas, descarta quem viola as regras (seção 4) e, entre os restantes, prioriza quem foi escalado menos vezes recentemente, para distribuir de forma justa. Gera um rascunho, que o líder revisa, ajusta e publica. Nunca publica sozinho. O resultado é previsível e explicável.

- **Camada 2, LLM (opcional, depois):** só para interpretar pedidos em texto ("monte a escala de domingo, evite repetir quem serviu no último mês") e explicar as escolhas. O LLM não escala de verdade: toda sugestão passa pelo regras.php.
- Se usar LLM: enviar IDs em vez de nomes e dados de contato (LGPD) e considerar o custo por chamada dentro do orçamento.

10. Outras ideias de uso de IA (priorizadas)

Nada desta seção é necessário no MVP; é uma lista para quando o básico estiver rodando (a importação dos dados atuais pode ser avaliada antes, se houver muitas planilhas). Princípio: só usar modelo de linguagem quando o algoritmo não resolve; é mais barato, previsível e explicável.

Com algoritmo (sem custo por uso)

1. **Sugestão de substituto na troca (prioridade alta):** lista quem pode cobrir, filtrando por bloqueios e regras e priorizando quem serviu menos recentemente.
2. **Alerta de desequilíbrio:** avisa o líder quando alguém serve muito mais ou menos que o restante do departamento.
3. **Risco de falta:** começar com regras simples (ex.: quem não confirmou até 48h antes) e sugerir reserva. Aprendizado de máquina só quando houver histórico. Usar apenas dados de presença (LGPD).

Com modelo de linguagem (custo por chamada, baixo se pontual)

4. **Importação dos dados atuais (prioridade alta no início):** ler planilhas, PDFs ou mensagens de grupo desorganizadas e transformar em cadastros estruturados (pessoa, departamento, data). O administrador confere antes de importar.
5. **Redação de comunicados:** o líder escreve 2 linhas e o sistema sugere a mensagem; o líder revisa e envia.
6. **Resumos para a administração:** os números vêm de consultas SQL; o modelo apenas escreve o relatório. Assim ele não inventa dados.
7. **Pedido em texto do líder** (camada 2 da seção 9).

Com cuidado: interpretar respostas livres no WhatsApp. Preferir botões de resposta rápida (Confirmar / Não posso), que dispensam IA e erram menos; as conversas abertas passaram a ser cobradas a partir de 01/10/2026.

11. Projeto de referência: "Escala 12" (software de escala de bombeiros)

O usuário achou este modelo "bem objetivo" e enviou o código como referência (stack diferente da nossa: React + NestJS + Prisma + PostgreSQL, em TypeScript). Não será reaproveitado como código, apenas como desenho. A IA que receber este briefing não tem acesso ao arquivo, a menos que o usuário o envie junto.

O que aproveitar

- **Gerador de escala guloso:** para cada dia, descarta quem está de férias, bloqueou a data ou teria plantão em dia seguido; entre os restantes escolhe quem tem menos plantões; dias sem ninguém disponível vão para uma lista de "não resolvidos" exibida ao administrador.
- **Métricas no painel:** cobertura (dias sem ninguém escalado), diferença entre quem mais e quem menos serve e nota de equilíbrio de 0 a 100 (baseada no desvio padrão).
- **Troca em duas etapas** (pedido, destinatário responde, aplica em transação, revalidando as regras no aceite).
- **Auditoria persistente** de todas as operações.
- **Estados da escala:** rascunho, enviada, aceita.

- **Validação de disponibilidade em um único método**, usado por geração, edição manual e troca.

Diferenças em relação ao nosso caso

- No projeto de referência há uma pessoa por dia (data é chave única). No nosso há várias pessoas por evento, em vários departamentos e igrejas, e a regra de dias seguidos vale por departamento.
- Lá, qualquer violação rejeita a troca na hora (o estado AGUARDANDO_ADMIN existe no enum, mas o fluxo não o usa). Aqui, violação leva à aprovação do líder.
- Notificações lá são internas e por e-mail, enviadas na hora. Aqui: fila no banco e WhatsApp.
- Lá o bloqueio guarda um motivo em texto. Aqui: apenas o período, por causa da LGPD.

Cuidados identificados no código de referência (não copiar)

- Regenerar a escala do ano apaga tudo e recria, perdendo trocas e edições manuais. No nosso sistema, regenerar deve preservar o que já foi confirmado.
- A regra de dia anterior/seguinte está repetida em vários pontos do código. Centralizar no regras.php.
- O README do backend admite que o código não foi executado no ambiente de origem e traz admin / admin123 como acesso padrão.
- Estilo muito compacto, difícil de manter para iniciantes.

Mapeamento para PHP puro

Projeto de referência	Nossa versão (PHP puro)
Prisma schema + migrations	Arquivo db/schema.sql (e arquivos de alteração numerados, ex.: db/001_nova_coluna.sql)
schedule.service.ts	Arquivos em src/ (regras.php, gerador.php, trocas_service.php)
Guards de papel (RBAC)	Função exigirPerfil('lider') chamada no início de cada página
AuditLog	Tabela auditoria, preenchida pela função registrarAuditoria() chamada nas telas e serviços
Notificações	notificar.php + tabela fila_envios + cron_avisos.php

12. Pontos ainda em aberto

1. Detalhes do fluxo de aprovação do líder nas trocas (quem aprova, prazo, o que ocorre se recusar).
2. Registro do número de WhatsApp que já está em uso no aplicativo (ver seção 6).
3. Escolha entre hospedagem compartilhada e VPS, e do provedor/integração de WhatsApp (comparar preços atuais).
4. Confirmar taxas em reais da API do WhatsApp com o provedor escolhido.
5. Definir se haverá, no futuro, algum caso em que o líder possa escalar mesmo violando regras (hoje só está definido o fluxo de aprovação na troca).
6. Confirmar com o usuário o fluxo sugerido de troca da seção 4 (validar no aceite e usar AGUARDANDO_LIDER).
7. Confirmar se a hospedagem compartilhada escolhida permite cron a cada minuto e a versão do PHP desejada.

13. Próximo passo sugerido

Montar o db/schema.sql com as tabelas do projeto (igrejas, departamentos, usuarios, departamento_usuario, eventos, escalas, bloqueios, trocas, auditoria, notificacoes, fila_envios), com índices e chaves estrangeiras. Em seguida, subir o Docker, criar o login (com password_hash, token CSRF e sessão) e a primeira tela (cadastro de igrejas e departamentos), explicando cada arquivo para quem está começando.